Lab 1 Report

112062119 楊竺函

1. Implementation

Proj02-02 Reducing the Number of Intensity Levels in an Image

```python
def reduceIntensityLevel(originalImage, intensityLevel):
    """
    originalImage: 還沒 quantize 的原影像
    intensityLevel: reduce 的 intensity level 數量 (2 到 256)
    quantizedImage: intensity level quantized 後的影像
    """
    if intensityLevel == 256:
        return originalImage

    # compute the size of a single step e.g. level = 4  -> each step is 64
    step = 256 // intensityLevel

    # create an empty image with equal size to the original image
    quantizedImage = np.empty_like(originalImage)

    h,w = originalImage.shape

    # compute the quantized value for each pixel
    for i in range(h):
        for j  in range(w):
            # new value = # of steps * size of step
            quantizedImage[i,j] = (originalImage[i,j] // step) * step

    # print(np.unique(quantizedImage))

    return quantizedImage
```

- The function reduceIntensityLevel(originalImage, intensityLevel) performs intensity quantization on a grayscale image.
- First, the step size between two adjacent intensity levels is computed by step = 256 // intensityLevel. This value determines the size of each quantization interval.
- An output image with the same dimensions as the original image is created using np.empty_like(originalImage).
- The program then iterates through every pixel in the image using two nested loops.
  - For each pixel (i, j), the quantized intensity value is computed using quantizedImage[i, j] = (originalImage[i, j] // step) * step
  - The operation originalImage[i, j] // step determines which quantization interval the pixel belongs to, and multiplying by step maps the value to the corresponding quantized intensity level.
  - The computed value is stored in the output image.
- After all pixels have been processed, the quantized image is returned as the result.

Proj02-03 Zooming and Shrinking Images by Pixel Replication

```python
def resizeImage_replication(originalImage, scalingFactor):
    """
    originalImage: 還沒 resize 的原影像
    scalingFactor: resize 的 scale (例如要放大為兩倍: 2; 縮小為一半: 1/2)
    resizedImage: resize 後的影像
    """
    # compute the size of resized image
    scale = float(scalingFactor)

    h,w = originalImage.shape

    new_h = max(1, round(h * scale))
    new_w = max(1, round(w * scale))

    # prepare a new empty image
    resizedImage = np.empty((new_h,new_w), dtype=originalImage.dtype)

    # preprocessing: find the nearest neighbor for each pixel
    src_rows = [min(int(i/scale), h-1) for i in range(new_h)]
    src_cols = [min(int(j/scale), w-1) for j in range(new_w)]

    # fill in each pixel with the value of its nearest neighbor from the original image
    for i in range(new_h):
        for j in range(new_w):
            # print(f"processing pixel ({i}, {j})...")
            resizedImage[i,j] = originalImage[src_rows[i], src_cols[j]]

    return resizedImage
```

- The function resizeImage_replication(originalImage, scalingFactor) performs image resizing using pixel replication (nearest neighbor interpolation).
- First, the scaling factor is converted to a floating-point value using scale = float(scalingFactor).
- The height and width of the original image are obtained using h, w = originalImage.shape.
- The dimensions of the resized image are then computed based on the scaling factor as
  - new_h = max(1, round(h * scale)), new_w = max(1, round(w * scale))
- A new empty image array with the computed dimensions is created using resizedImage = np.empty((new_h, new_w), dtype=originalImage.dtype)
- To improve efficiency, the nearest source pixel indices for each row and column are precomputed:
  - src_rows = [min(int(i / scale), h - 1) for i in range(new_h)]
  - src_cols = [min(int(j / scale), w - 1) for j in range(new_w)]
- The program iterates through every pixel in the resized image using nested loops.
  - For each pixel, the value of the nearest pixel from the original image is copied to the resized image:
    resizedImage[i, j] = originalImage[src_rows[i], src_cols[j]]
- After all pixels have been processed, the resized image is returned as the result.

Proj02-04: Zooming and Shrinking Images by Bilinear Interpolation

```python
def resizeImage_bilinear (originalImage, scalingFactor):
    """
    originalImage: 還沒 resize 的原影像
    scalingFactor: resize 的 scale (例如要放大為兩倍: 2; 縮小為一半: 1/2)
    resizedImage: resize 後的影像
    """

    # create a new image(numpy array) with adjusted size
    scale = float(scalingFactor)

    h,w = originalImage.shape

    new_h = max(1, round(h * scale))
    new_w = max(1, round(w * scale))

    resizedImage = np.empty((new_h,new_w), dtype=originalImage.dtype)

    ## preprocessing

    # compute the mapping coordinate for each pixel
    src_ys = np.arange(new_h) / scale
    src_xs = np.arange(new_w) / scale

    # compute the coordinate of four neighbors
    y1s = np.floor(src_ys).astype(int)
    x1s = np.floor(src_xs).astype(int)
    y2s = np.minimum(y1s + 1, h - 1)
    x2s = np.minimum(x1s + 1, w - 1)

    # compute the difference on both axes for interpolation
    dys = src_ys - y1s
    dxs = src_xs - x1s

    # main loop to calculate bilinear interpolation result for each pixel
    for y in range(new_h):

        if y % 500 == 0:
            print(f"processing row {y}..")

        for x in range(new_w):

            y1 = y1s[y]
            x1 = x1s[x]

            y2 = y2s[y]
            x2 = x2s[x]

            dx = dxs[x]
            dy = dys[y]

            # the formula of interpolation
            new_value = (1-dx) * ((1-dy)* originalImage[y1,x1] + dy * originalImage[y2,x1])
                        + dx * ((1-dy) * originalImage[y1,x2] + dy * originalImage[y2,x2])

            # fill in the computed new value
            resizedImage[y, x] = np.clip(round(new_value), 0, 255)

    return resizedImage
```

- The function resizeImage_bilinear(originalImage, scalingFactor) resizes an image using bilinear interpolation.
- The computation of the scaling factor, the dimensions of the resized image, and the creation of the output image array are the same as in the previous problem and are therefore omitted here.
- For each pixel in the resized image, the corresponding coordinate in the original image is first computed using inverse mapping:
  - src_ys = np.arange(new_h) / scale
  - src_xs = np.arange(new_w) / scale
- The integer coordinates of the four neighboring pixels surrounding the mapped location are then calculated:
  - y1 = floor(src_y), x1 = floor(src_x)

- y2 = min(y1 + 1, h - 1), x2 = min(x1 + 1, w - 1)

- The fractional distances between the mapped coordinate and its upper-left neighbor are computed as
  - dy = src_y - y1
  - dx = src_x - x1
- These values represent the relative position of the mapped point within the surrounding pixel grid.
- The intensity value of the new pixel is then computed using the bilinear interpolation formula, which combines the four neighboring pixels:

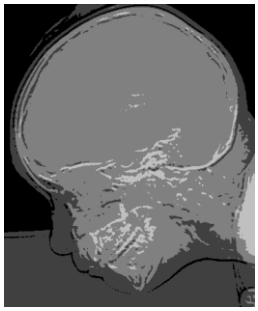
$$v = (1-dx)((1-dy)f(x1,y1) + dy\ f(x1,y2)) + dx((1-dy)f(x2,y1) + dy\ f(x2,y2))$$

- The computed value is clipped to the valid intensity range [0, 255] and stored in the resized image
- After all pixels have been processed, the resized image is returned as the result.

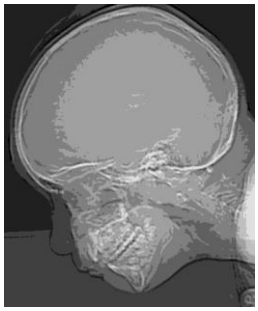
2. Execution Result

Proj02-02 Reducing the Number of Intensity Levels in an Image

Intensity Levels = 2

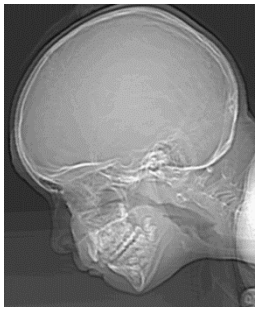


Intensity Levels = 4



Intensity Levels = 8

Intensity Levels = 16



Intensity Levels = 32

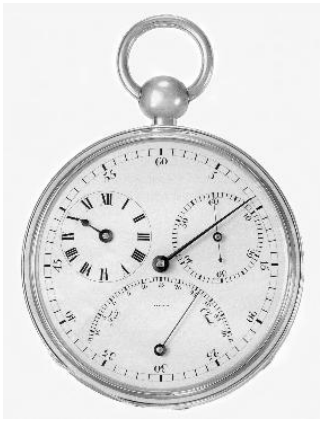
Intensity Levels = 64

Intensity Levels = 128

Intensity Levels = 256

Proj02-03 Zooming and Shrinking Images by Pixel Replication

(a) original image



(b) shrink (a) by 10x

(c) zoom (b) by 10x

Proj02-04: Zooming and Shrinking Images by Bilinear Interpolation

(a) original image (1250dpi)

(b) shrink (a) to 100dpi

(c) zoom (b) back to 1250dpi

3. Discussion

Proj02-03 Zooming and Shrinking Images by Pixel Replication

When the image is shrunk by a factor of 10, the algorithm performs subsampling, which means that many pixels from the original image are discarded. As a result, significant image information is lost during the shrinking process.

When the reduced image is zoomed back to the original resolution using pixel replication, each pixel is simply replicated to fill a larger area. However, pixel replication does not create new information; it only copies existing pixel values. Therefore, the lost details from the shrinking step cannot be recovered.

As a result, the final image becomes blurrier at object boundaries and contains fewer details compared to the original image.

Proj02-04: Zooming and Shrinking Images by Bilinear Interpolation

When the image is shrunk from 1250 dpi to 100 dpi, many pixels are removed during the down sampling process. Because of this, some image information is lost, especially small details and sharp edges. Once this information is lost, it cannot be fully recovered when the image is enlarged again.

When the image is zoomed back to 1250 dpi using bilinear interpolation, the new pixel values are calculated from the four neighboring pixels. This method makes the image look smoother and reduces the blocky appearance that may occur during scaling. However, because it averages neighboring pixels, the edges in the image become less sharp and appear slightly blurred compared with the original image.

If we compare this result with the image obtained using nearest neighbor interpolation, the difference becomes clearer. Nearest neighbor interpolation tends to keep edges sharper because it simply copies the closest pixel value. However, it often creates visible blocky artifacts. In contrast, bilinear interpolation produces smoother transitions between pixels, but the edges may look more blurred.